



Fundusze
Europejskie
Wiedza Edukacja Rozwój



Rzeczpospolita
Polska

Politechnika
Warszawska

Unia Europejska
Europejski Fundusz Społeczny

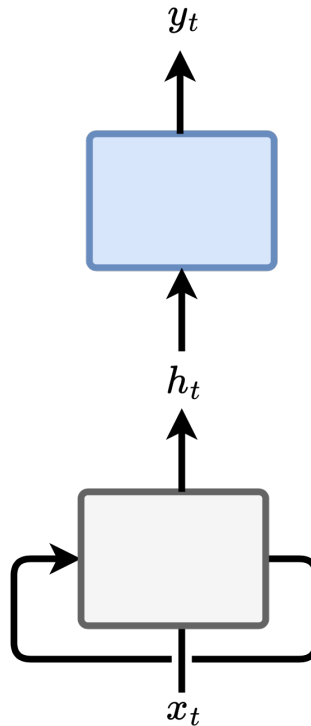


Algorytmy w Inżynierii Danych

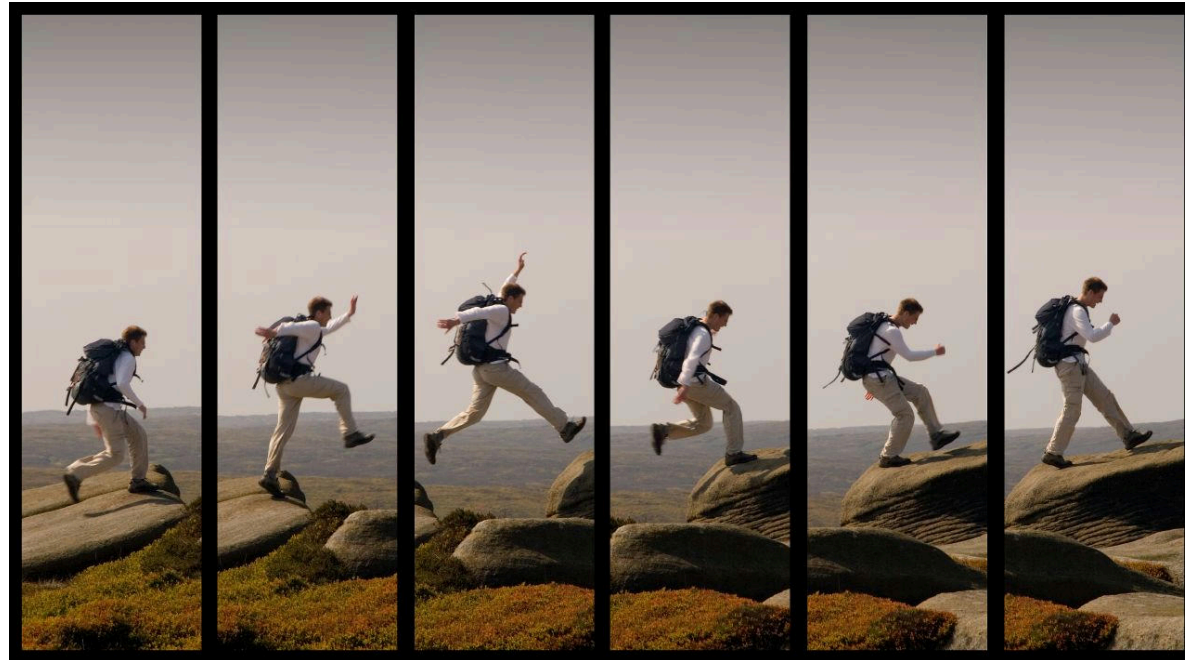
Rekurencyjne Sieci Neuronowe

Bartosz Chaber

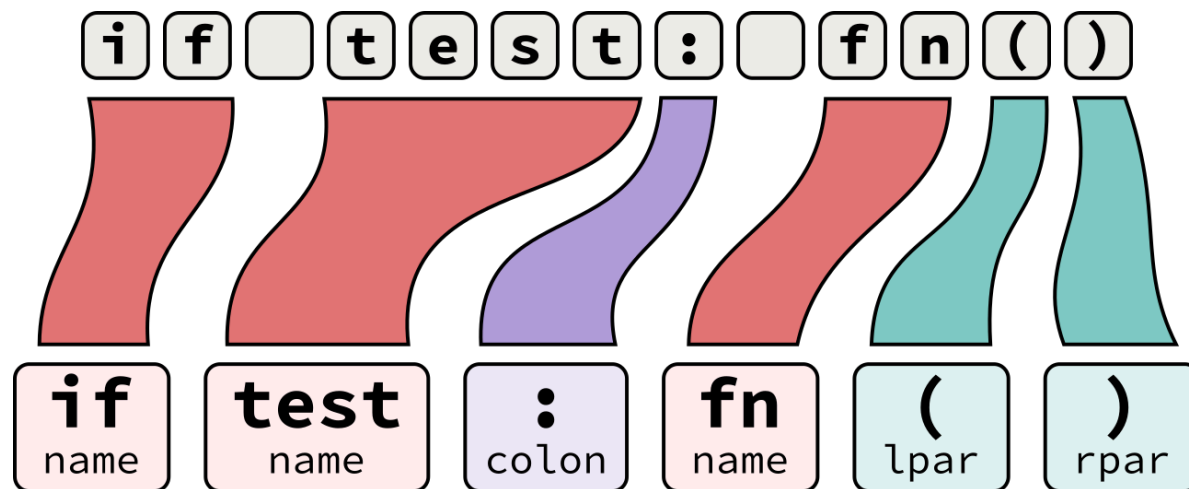
W skrócie RNN – ang. Recurrent Neural Networks.



Podczas, gdy naturalnym rodzajem danych przetwarzanym przez sieci konwolucyjnych są **wielokanałowe obrazy**, to sieci rekurencyjne operują na **sekwencjach danych**.

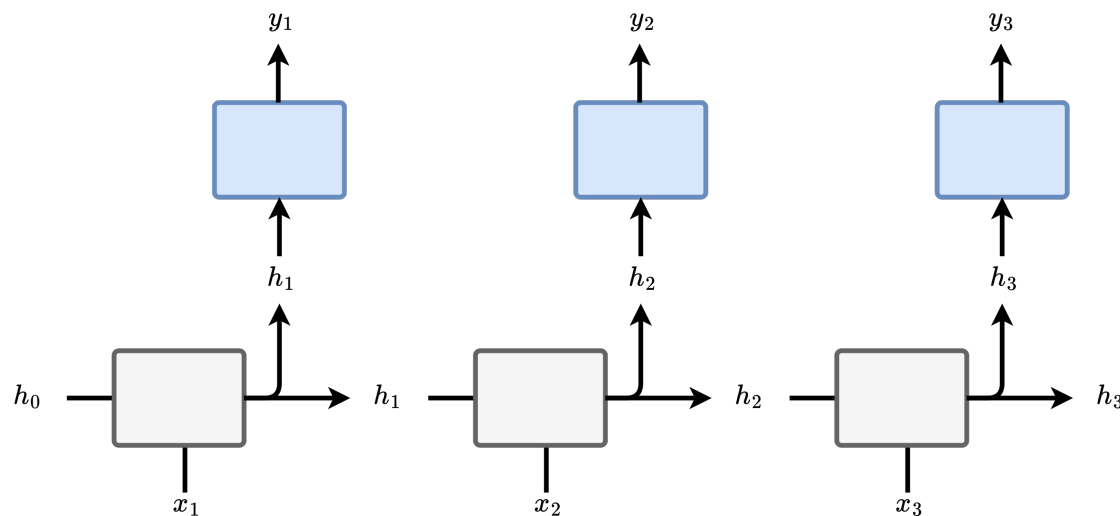


Przetwarzanie języka naturalnego jest domeną sieci rekurencyjnych, z uwagi na wykorzystanie **kolejności** kolejnych elementów zdań.

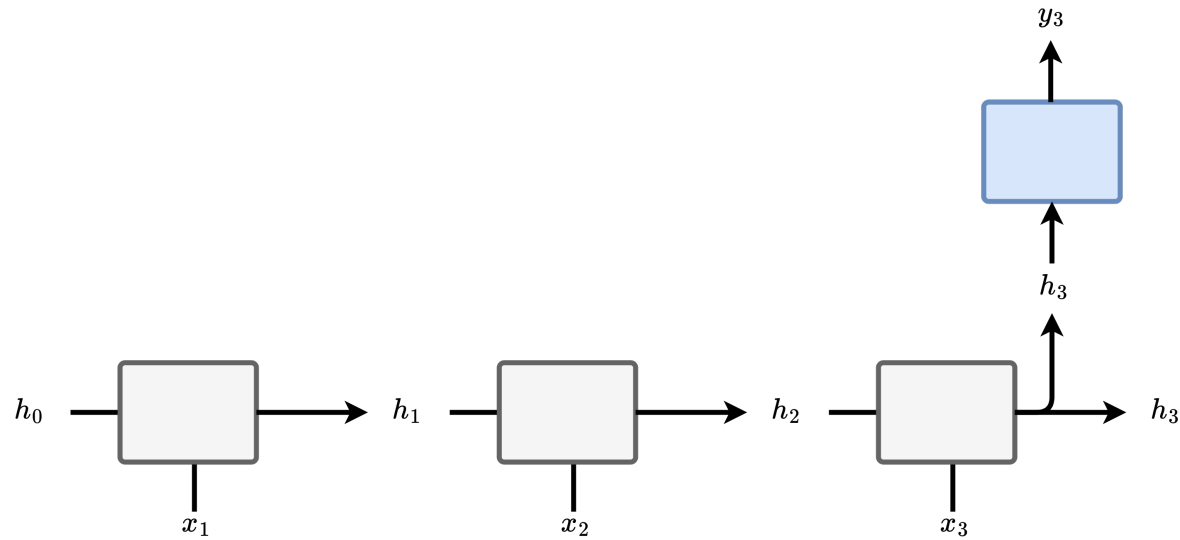


W zależności od zastosowania, możemy spotkać się z różnymi architekturami sieci rekurencyjnych.

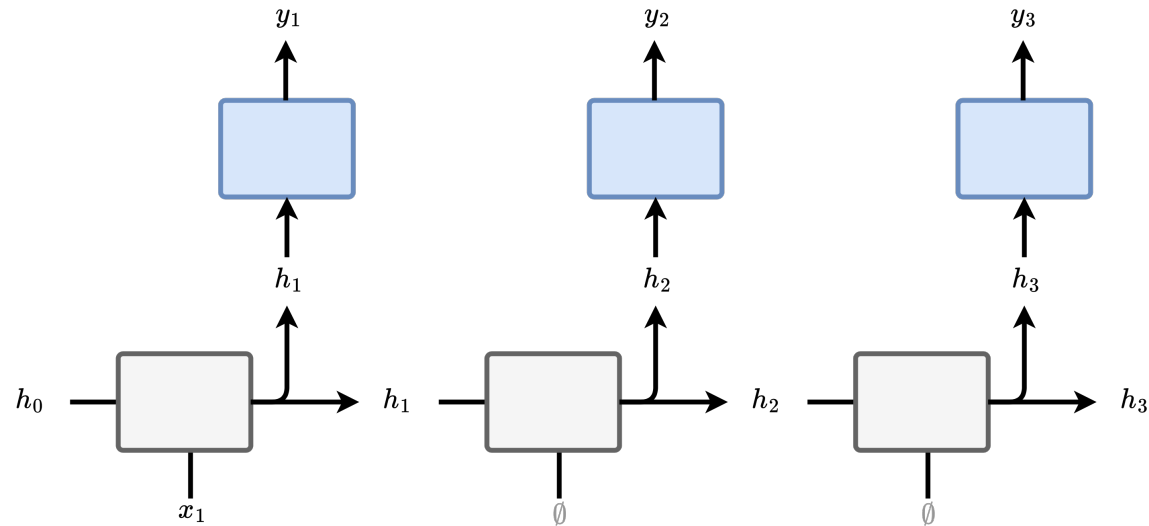
Przykładowo, w tłumaczeniu między językami, popularna jest architektura wiele-wejść, wiele-wyjść:



Klasyfikacja sekwencji najczęściej realizowana jest w architekturze wiele-wejść, jedno-wyjście:



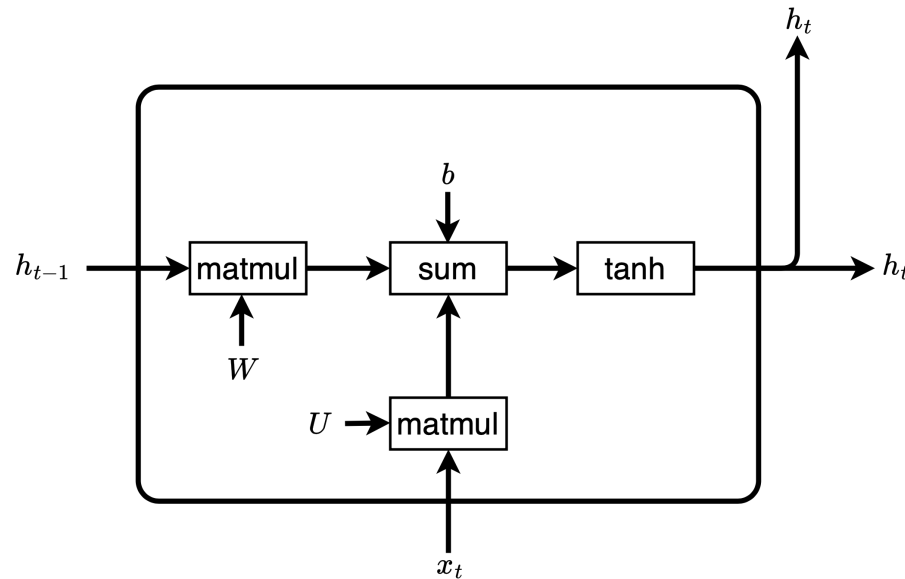
Innym zastosowaniem sieci RNN jest generacja podpisów podanego obrazu. Wtedy sieć działa jak generator, w trybie jedno-wejście, wiele-wyjść:



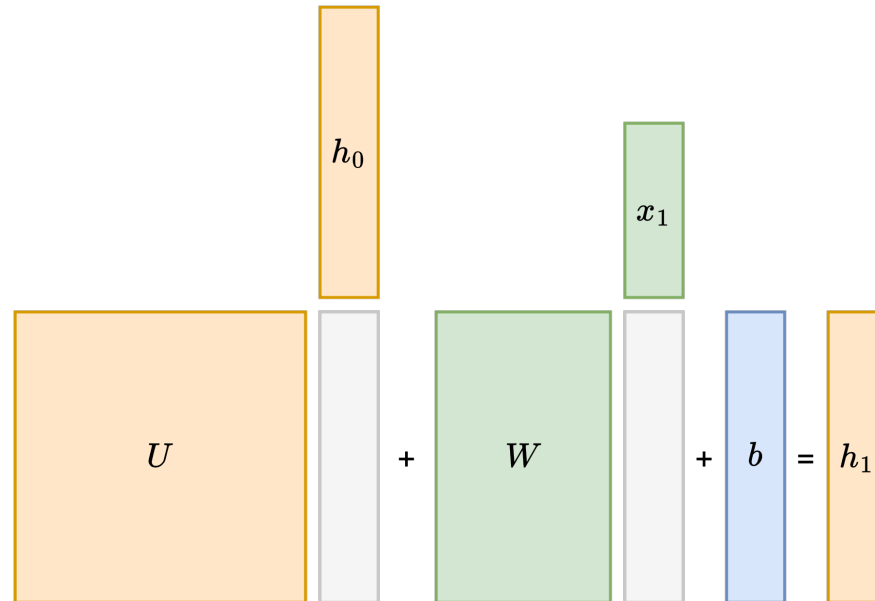
Najprostsza wersja pojedynczej komórki RNN to tzw. *vanilla RNN cell*.
Macierze U i W są **współdzielone** przez komórki *rozwiniętej* sieci rekurencyjnej.



Najprostsza wersja pojedynczej komórki RNN to tzw. *vanilla RNN cell*.
Macierze U i W są **współdzielone** przez komórki *rozwiniętej* sieci rekurencyjnej.



Ukryte stany h_t są wektorami o pewnej długości i służą jako pamięć, pozwalająca wspomóc analizę aktualnego elementu sekwencji. Początkowy stan h_0 jest zwykle wektorem zerowym.



Sposób kodowania elementów sekwencji jest pewnym wyborem architekta sieci. Jak reprezentować w postaci wektorów liczb, np. słowa w zdaniu? Istnieją już opracowane tzw. techniki *word embedding*¹, np. Words2Vec czy GloVe. Najbardziej prymitywnym sposobem jest opracowanie słownika ze znanymi słowami i przypisanie słowom poszczególnych indeksów.

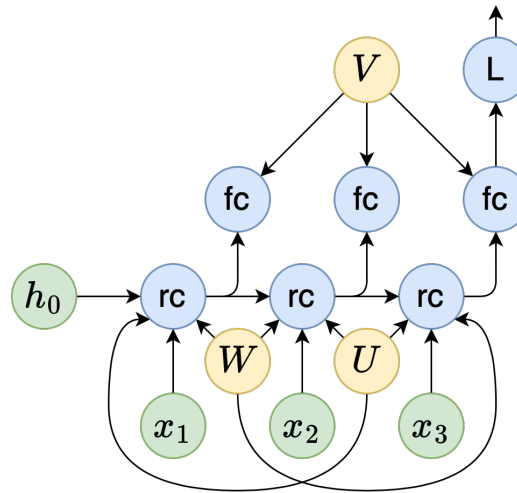
¹zagnieżdżania słów?

Przykład: znając tylko trzy słowa (“mała”, “bardzo”, “żaba”) można każdemu przypisać indeks (1 - “mała”, 2 - “bardzo”, 3 - “żaba”).

Wtedy każde z tych słów można zapisać przy pomocy trzy elementowych wektorów (tak jak np. kodowane są klasy na wyjściu sieci).

Zdanie “bardzo bardzo mała żaba” można zakodować jako cztery wektory: $[0, 1, 0]^T$, $[0, 1, 0]^T$, $[1, 0, 0]^T$, $[0, 0, 1]^T$.

Ograniczając się do stałej i niedużej liczby elementów w sekwencji możemy statycznie rozwinąć rekurencję powielając komórki rekurencyjne. Dla przykładu, sieć rekurencyjna typu wiele-wejść, jedno-wyjście, akceptująca sekwencje złożone z trzech elementów.



Z uczeniem sieci rekurencyjnych skojarzony jest termin **Backpropagation Through Time**, natomiast jest to ten sam algorytm, który wykorzystuje się w nierekurencyjnych sieciach neuronowych.

Z uczeniem sieci rekurencyjnych skojarzone są dwa problemy: zanikający gradient oraz eksplodujący gradient.

Zanikający gradient polega na tym, że propagacja w przejściu w tył przez kolejne funkcje nieliniowe (np. tanh) skutkuje wygaszaniem wartości gradientu przy długich sekwencjach.

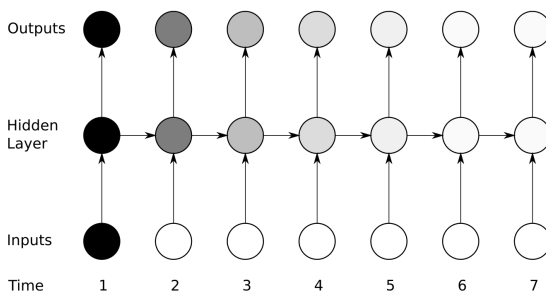


Figure 4.1: **The vanishing gradient problem for RNNs.** The shading of the nodes in the unfolded network indicates their sensitivity to the inputs at time one (the darker the shade, the greater the sensitivity). The sensitivity decays over time as new inputs overwrite the activations of the hidden layer, and the network 'forgets' the first inputs.

źródło: <https://doi.org/10.1007/978-3-642-24797-2>

Rozwiązaniem jest np. zastosowanie innych komórek rekurencyjnych, np. LSTM (Long-Short Term Memory) czy GRU (Gated Recurrent Unit), które są w stanie propagować gradient bez zbędnego tłumienia.

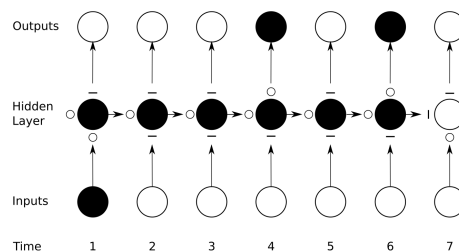


Figure 4.4: **Preservation of gradient information by LSTM.** As in Figure 4.1 the shading of the nodes indicates their sensitivity to the inputs at time one; in this case the black nodes are maximally sensitive and the white nodes are entirely insensitive. The state of the input, forget, and output gates are displayed below, to the left and above the hidden layer respectively. For simplicity, all gates are either entirely open ('O') or closed ('—'). The memory cell 'remembers' the first input as long as the forget gate is open and the input gate is closed. The sensitivity of the output layer can be switched on and off by the output gate without affecting the cell.

źródło: <https://doi.org/10.1007/978-3-642-24797-2>

Z drugiej strony, możemy spotkać się z eksplozją gradientu. W efekcie, wagi sieci mogą przekroczyć dopuszczalny zakres wartości (ich wartość staje się NaN czyli *Not a Number*). Rozwiązaniem jest sztuczne ograniczanie wartości gradientów (np. od -5 do $+5$). Dobrze jest też zastosować bardziej wyszukaną metodę inicjalizacji wag, np. metodę Xavier.

W trybie generacji, po nauczaniu sieci, na wyjściu sieci zwracane jest prawdopodobieństwo kolejnego elementu sekwencji. Możliwe jest rozszerzenie funkcji softmax o parametr nazwany temperaturą. Im większa, tym bardziej “rozmyte” jest prawdopodobieństwo.

$$\sigma(z, T)_i = \frac{e^{\frac{z_i}{T}}}{\sum_{j=1}^K e^{\frac{z_k}{T}}}$$

```
julia> import Flux: softmax
julia> softmax([-1.0, 123.0, 0.0, 0.1] ./ 1.0)
[0.0, 1.0, 0.0, 0.0]
julia> softmax([-1.0, 123.0, 0.0, 0.1] ./ 10.0)
[0.000004, 0.999987, 0.000004, 0.000004]
julia> softmax([-1.0, 123.0, 0.0, 0.1] ./ 100.0)
[0.1544, 0.5335, 0.1560, 0.1561]
```

Literatura

- Alex Graves, Supervised Sequence Labelling with Recurrent Neural Networks, 2012, ISBN: 978-3-642-24797-2
- Tomáš Mikolov, Statistical Language Models Based on Neural Networks, 2012, Brno University of Technology, Faculty of Information Technology, rozprawa doktorska,
- Andrej Karpathy, The Unreasonable Effectiveness of Recurrent Neural Networks, 2015, URL: <http://karpathy.github.io/2015/05/21/rnn-effectiveness/>

- Aditya Agrawal, Back Propagation in Recurrent Neural Networks, 2018, URL: https://www.adityaagrawal.net/blog/deep_learning/bprop_rnn
- Denny Britz, Recurrent Neural Networks Tutorial, Part 3 – Backpropagation Through Time and Vanishing Gradients, 2015, URL, <https://dennybritz.com/posts/wildml/recurrent-neural-networks-tutorial-part-3/>

Dziękuję za uwagę